UriNet

Build the simplest modular blockchain for real-life utility: a network of plug-and-play, single-purpose chains—natively interconnected and anchored by a common core: UriNet—sharing universal payments, built-in security, and live data so products just work.

Ryan Kwak ryankwak@uriverse.com
Jun Kwak jkwak@uriverse.com
Boram Kim bokim@uriverse.com

NOTE: By accessing or reading this whitepaper, you acknowledge and agree to the full disclaimer provided at the end of this document. You must review and understand the disclaimer in its entirety before proceeding with the rest of the whitepaper.

Abstract

UriNet is a modular utility network designed for real-world products. Each application runs on its own sovereign Utility Chain, natively connected to a high-assurance mainnet (UriChain) and surfaced through a common gateway (UriPortal). Shared rails—universal payments, built-in security, identity/compliance, data/oracles, interchain messaging, and a built-in DEX—are provided once at the core and reused everywhere, so assets and actions move seamlessly across chains. The URI utility token powers fees, staking, and access to services, enabling predictable, URI-denominated economics across the ecosystem. The architecture targets low-latency performance (≤500 ms blocks, ~10k TPS per chain), offers flexible security (validator-lease and checkpointed options), and launches EVM first with a Rust environment to follow. Governance spans core protocol changes, shared resource allocation, a Foundation-run Utility Chains Manager, and UriPortal standards. The result is web-simple UX, composability by default, and immediate fit for use cases such as RWA/STO platforms, payments and loyalty, ads

and media integrity, insurance/oracles, gaming, philanthropy, and civic tech—delivered via a phased roadmap from public testnets to mainnet, official utility chains, and open ecosystem standards.

Introduction

The past decade proved blockchains can be decentralized computers, but not everyday infrastructure: monoliths trade throughput and latency for generality, liquidity fragments behind brittle bridges, and most apps orbit speculation instead of real use. UriNet is a purpose-built answer—a modular utility network where each real-world product runs on its own sovereign Utility Chain, natively interconnected and anchored by the high-assurance UriChain mainnet. Shared services—universal payments, built-in security, identity/compliance, oracles, data, and a built-in DEX—are provided once at the core and surfaced through UriPortal, while native interchain messaging moves assets and actions seamlessly across chains and into the physical economy. Interoperability and composability are defaults, so builders ship web-simple experiences without wrestling the machinery. URI, the network's native utility token, powers access to shared services and on-chain operations across UriNet. In short: not a blockchain looking for use cases, but the blockchain ecosystem real-world applications have been waiting for.

What's UriNet?

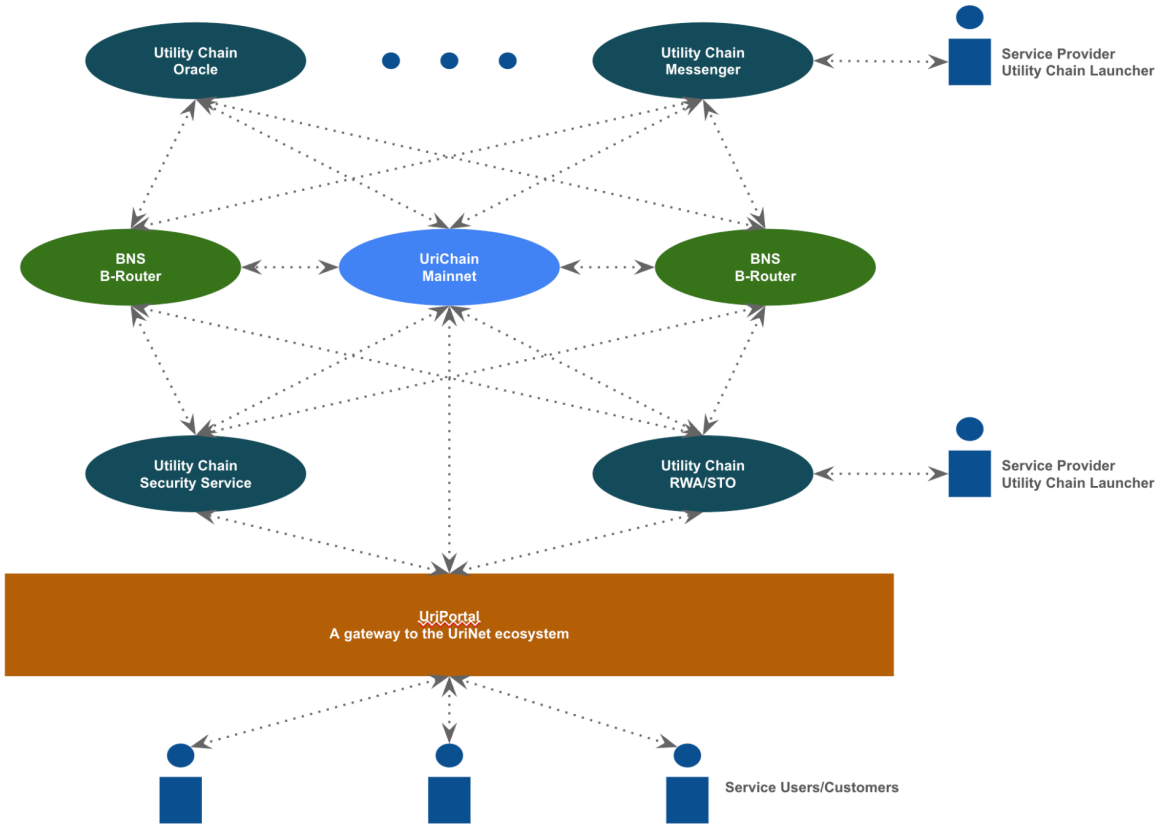
UriNet is a modular utility network: a system of sovereign, single-purpose Utility Chains that are natively interconnected and anchored by a high-assurance mainnet, UriChain. Instead of forcing every product into a one-size-fits-all blockchain, UriNet lets each application run its own chain while sharing common services—universal payments, built-in security, identity/compliance, data/oracles, comms/social, a built-in DEX, and universal discovery—exposed through UriPortal. Native interchain messaging makes assets and actions move seamlessly across chains and into the physical economy, so applications can cooperate by default rather than bolt on bridges after the fact. URI, the network's native utility token, powers access to these services and on-chain operations across UriNet.

Core components

- **UriChain (mainnet hub):** High-assurance base for shared security, settlement, universal payments, and network-wide services.
- **Utility Chains (sovereign branches):** Single-purpose chains dedicated to one product or service, with full control over logic and cadence, while inheriting interoperability and shared services.
- **UriPortal (ecosystem gateway):** A common entry point for discovery, onboarding, and APIs—so users and builders find, use, and compose services without stitching infrastructure themselves.
- **Native Interchain Messaging:** First-class, protocol-level communication so chains coordinate state and liquidity without brittle, opt-in bridges.
- **URI token:** The standard unit of utility across UriNet—used to access, pay for, and participate in all network services and shared modules; also staked to secure the network and for governance.

Design principles

- **Application-first:** Start from real products and user needs; pick modules and launch a chain in minutes.
- **Sovereign yet harmonized:** Each chain governs itself but speaks the same language and shares core rails.
- **Composability by default:** Services are reusable across chains; cooperation is the baseline, not an afterthought.
- **Web-simple UX:** Predictable costs, progressive security, and familiar flows delivered through UriPortal.



< UriNet >

Why UriNet?

Most networks either centralize everything on one chain or scatter apps across sidechains and rollups that rarely interoperate in practice. The result: fragmented liquidity, brittle bridges, uneven security, and UX that never feels like the web. Large partners avoid the complexity; smaller teams struggle to ship and scale. UriNet exists to solve for **real-life utility**—products people use daily—by aligning architecture with how applications are actually built and run.

The problems we address

1. **One-chain-fits-all doesn't fit:** General-purpose chains force conflicting tradeoffs in fees, latency, and governance.
2. **"Opt-in" interoperability fails in practice:** Ad-hoc bridges and incompatible standards make cooperation exceptional, not default.
3. **Liquidity and trust fragment:** Value gets stranded across execution silos; security and compliance vary by island.
4. **Builders fight infrastructure, not problems:** Teams reinvent payments, identity, and data feeds instead of shipping features.
5. **Real-world products need shared rails:** RWA/STO, loyalty/benefits, ads/payments, news/media integrity, and insurance oracles require common compliance, settlement, and data—delivered reliably.

Our approach

- **Application-first architecture:** Start from the product and give it a dedicated Utility Chain. No shoehorning into global mempools or fee markets.
- **Shared services at the core:** Payments, security, identity/compliance, data/oracles, comms/social, DEX, and discovery are provided once on UriChain and consumed everywhere via UriPortal.
- **Native interchain messaging:** Cooperation is built in—assets and actions move safely and predictably across chains without bespoke bridges.
- **Sovereignty with guardrails:** Each chain controls its logic and pace while inheriting baseline security, tooling, and interoperability.
- **Predictable economics:** URI standardizes costs and incentives across chains so builders can model, monetize, and grow.

Who it's for (initial ecosystem initiative)

- **RWA / Security-token platforms:** Unified identity/compliance and settlement rails with native cross-chain distribution.

- **Advertising and payments:** Verifiable attention and instant, programmable payouts across services.
- **News and media:** Authenticity, provenance, and monetization modules that travel network-wide.
- **Benefits and loyalty:** Interoperable rewards that can be earned in one service and redeemed in another.
- **Insurance and risk:** Oracle and claims workflows that span multiple providers and data sources.

What this enables

- **Web-simple experiences:** Users onboard once, discover via UriPortal, and move value across services without friction.
- **Composability that compounds:** Every new chain adds reusable modules to the whole network.
- **Credible neutrality for partners:** Shared rails and standards invite “big-guy” participation instead of pushing it away.
- **Faster shipping, lower ops burden:** Teams focus on products; UriNet handles the plumbing.

In short, UriNet is not a blockchain looking for problems—it is an application-first, harmonized ecosystem created because real products needed a better foundation.

Design Principles

UriNet is engineered for real-life utility. The principles below guide every architectural choice and product decision. They frame what we optimize for; the following System Architecture section will cover how we realize them.

1. Application-first

Real products come first. Each service runs on its own sovereign Utility Chain with the minimum set of modules needed to launch fast, iterate safely, and serve a specific use case. UriPortal exposes shared rails so teams focus on features, not plumbing.

2. Modularity and composition (mix-and-match)

Everything is a module: payments, identity/compliance, data/oracles, comms/social, security, DEX, discovery. Builders assemble chains by plugging in prebuilt modules and can compose services provided by other chains. Modules are replaceable and upgradable without forking entire stacks.

3. Sovereign yet harmonized

Chains govern themselves—logic, cadence, and opt-in policies—while speaking a common protocol and inheriting core rails from UriChain. This “independent but compatible” model preserves local control without sacrificing network-wide coherence.

4. Interoperability by default (freedom of asset and action movement)

Assets and actions must move predictably across chains. Native interchain messaging and common asset standards make cross-chain transfers, calls, and settlements first-class—not ad-hoc bridges. Liquidity and state are portable across the network.

5. One for all, all for one

Security, payments, discovery, and other common services are provided once at the core and consumed everywhere. As new chains join, they strengthen the whole—adding features, data, and reusable services that every participant can benefit from.

6. Born-harmonized (synergy from day one)

Interoperability isn't bolted on later. URI-denominated fees, common identity primitives, and standard interfaces create immediate network effects: every new chain is instantly discoverable, composable, and settlement-compatible with the rest of UriNet.

7. Fast by design

User experience drives performance targets: low-latency confirmation paths, predictable finality, and throughput that scales with demand. All this while still aiming for under 500ms block times, enabling thousands of transactions per second, per chain—for near limitless aggregate TPS. We optimize end-to-end latency—from user action to settled state—so apps feel web-simple and responsive.

8. Elastic scalability

Scale horizontally by adding Utility Chains; scale vertically by tuning modules per chain. The network grows with usage rather than forcing all activity into a single global mempool or fee market. Capacity expands where the demand is.

9. Predictable economics

URI provides a consistent meter for compute and shared services across chains. Costs are low, transparent, and modelable; incentives are aligned so validators, builders, and users can plan, price, and grow without fee whiplash.

10. Credible neutrality, security, and compliance as primitives

The core rails aim to be neutral and widely accessible. Baseline security practices and identity/compliance modules exist for those who need them, without imposing one-size-fits-all policy on sovereign chains. Participation should be open, sustainable, and resistant to capture.

11. Simplicity over cleverness

Prefer simple, auditable mechanisms to theoretically flawless but fragile/convoluted designs. Clear interfaces, minimal surprises, and operability trump novelty—because real-world systems must be easy to reason about and easy to run.

Together, these principles ensure UriNet remains scalable, fast, and modular—freeing assets and actions to move wherever they create the most value—while keeping the ecosystem harmonized so every new chain strengthens the whole.

System Architecture

UriNet is a three-layered utility network composed of (1) the UriChain mainnet, (2) a constellation of sovereign Utility Chains, and (3) shared Ecosystem Infrastructure that onboards, catalogs, and connects everything. Together, they provide connectivity, settlement, payments, identity/compliance, data, and messaging as native capabilities rather than bolt-ons.

UriChain (Mainnet)

UriChain is the shared core for connectivity, coordination, and settlement across UriNet. It provides universal payments and assurance for the ecosystem, exposes a governance DAO, and includes a built-in DEX to enable currency swaps among Utility Chains. URI is the network's native utility token.

Utility Chains

Utility Chains are sovereign, single-purpose blockchains that plug into UriChain for payments, security options, interchain messaging, and discovery. Two categories are supported:

1. **Foundation/Community Essential Chains** – baseline services operated by the foundation/community: Security Service Chain, Oracle Service Chain, Message Service Chain, Social Community Service Chain, plus optional standalone private chains for regulated financial use cases under local law.

2. **General-Purpose, Product-Specific Chains** – purpose-built chains created by anyone; some may be promoted and officially managed by the Foundation/DAO. Typical examples include RWA/STO, community, donation, and ads.

Common characteristics

- **Sovereignty with shared rails:** Each chain is sovereign but taps UriChain's universal payments and the built-in DEX for liquidity.
- **Local tokenomics:** Chains may price in URI-pegged denominations for predictable UX or issue their own native tokens with independent economics.
- **Composable build model:** Builders can (a) mix and match native modules (validation, universal payments, chat, social, compliance) or (b) compose services from other chains via built-in interchain messaging—while remaining secured/connected through the mainnet.
- **Single-service scope:** Recommended to serve one application/service per chain for clarity and reliability.
- **Staking and Fees:** Some services require staking in URI on UriChain and/or fee payments to access
- **Security options:** Either run own validator set or rely on UriChain validators operating the security service chain.
- **Join/registration:** Chains register with the mainnet at launch (or later) to unlock ecosystem benefits: access to other service chains, larger economic surface area, and easier service construction.

Ecosystem Infrastructure (UriPortal)

UriPortal is the permissionless, open-API gateway into UriNet. It lets anyone discover services, register a chain, and connect to the ecosystem; staking URI is required to join to deter bad actors. The foundation/community maintains these open-source rails for the benefit of every Utility Chain.

Core components include:

- **Chain Registry:** Canonical on-chain directory of Utility Chains (owner/foundation, token/denom, endpoints, declared services).
- **Services API (Gateway):** Standard interfaces/schemas for exposing and consuming services, including registration, versioning, and discovery.
- **Discovery Portal:** Search/browse chains and capabilities with metadata, uptime, and integration docs.
- **Tokenomics & Analytics:** Comparable stats (supply, staking, fees, emissions, usage) and ecosystem-level insights.
- **Interchain Digital-Asset Transfer:** Native asset movement across chains, with routing/compatibility guided by the Registry and Services API.
- **Ecosystem DEX:** Swap assets issued by any Utility Chain; URI can serve as the common quote/settlement asset.
- **Oracle Application:** Curated external data feeds and attestations made available network-wide via the Services API; request-driven with a marketplace model.

Network Services & Cross-Chain Interoperability

UriNet provides **native interchain messaging**—an any-message, RPC-like protocol with cross-chain calls, state proofs, and acknowledgements—plus universal payments and standardized cross-chain token semantics. Fees and settlement are URI-denominated to deliver predictable costs.

URI Token

URI is UriNet's native utility token and UriChain's native coin. It is the network's standard unit for access and payment—used to execute transactions, cross-chain messaging, compute/storage, data/oracle requests, advertisements, identity/compliance, security leases, and shared modules (payments, routing, DEX). Participants stake URI to provision and secure services and to

participate in governance. All Utility Chains settle fees in URI, creating a single, predictable economic standard.

Consensus

UriChain (mainnet) uses a Modified Byzantine Fault-Tolerant Delegated Proof-of-Stake (DPoS) design. Utility Chains can choose among modes that fit their purpose (e.g., Modified BFT-PoS or PoA) to balance decentralization and speed.

Smart Contracts

Smart contracts are first-class and especially critical for financial services such as RWA, STO, NFT, Trust, etc. The rollout prioritizes EVM compatibility first, followed by a Rust-based smart-contract environment.

Technology: How UriNet Works Under the Hood

DISCLAIMER: As stated in our primary disclaimer, the technical details in this section are preliminary, may be incomplete or inaccurate, and are subject to change without notice. This section in particular is under active development and may be wrong; do not rely on it as advice, an offer, or a commitment. In any case of inconsistency, the primary disclaimer governs and supersedes this section.

This section describes UriNet's internal mechanics at a basic level.

Nomenclature and Roles

Networks :

- **UriNet:**The entire ecosystem (UriChain + Utility Chains + UriPortal). A modular, three-plane system designed for real-world utility.

- **Control & Settlement Plane (UriChain mainnet):** shared security, canonical routing/settlement, governance, and a built-in DEX. All network fees clear in **URI** for predictable costs.

- **Execution Plane (Utility Chains, "U-Chains"):** sovereign single-purpose chains that run product logic and interoperate natively through a standardized messaging and asset framework.

- **Interface & Coordination Plane (UriPortal):** open APIs, on-chain registry, discovery/analytics, onboarding, and enforcement of network-level SLAs, quotas, and reputations.

Together these planes make assets and actions move safely and predictably across chains while keeping builder UX web-simple and costs denominated in URI.

Actors

- **Validators***:* UriChain validators (URI-staked) that may opt into operating Security Service Chains and DAO committees.

- **BNS (Blockchain Name Service) + Blockchain Router**: Similar to a traditional DNS—these permissionless agents serve as a routing board for interchain data packets and proofs.

- **Reporters/ Oracles**: Nodes that fetch off-chain data and submit aggregated attestations.

- **Service (application) Providers**: U-Chain maintainers who may pay fees to UriNet and define SLAs.

- **Users**: End users and dApps that submit transactions to any chain.

Identifiers and Types (canonical)

```
 ChainID, Address, Denom, Amount(uint256), Height(uint64), Time(uint64
unix), URI (the native asset), Hash32/Hash256, Signature, PubKey,
Bytes, etc.
```

1. UriChain Core

UriChain provides: (i) universal payments, (ii) canonical interchain coordination and settlement, (iii) a governance DAO, and (iv) registry-anchored identity and routing. Core system services—**Payments Router, DEX Router, Messaging Hub, Chain Registry, Governance**—coordinate cross-chain execution and denominate fees in URI.

1.1 Consensus and Validator Set

UriChain runs a Modified BFT-DPoS: URI-staked validators produce blocks; delegators stake to validators. Epochs (e.g., 1–3 hours) determine validator set updates and reward/penalty settlement.

Safety/Liveness thresholds

- **Safety:** Byzantine tolerance f < n/3 within an epoch.
- **Liveness:** progresses if ≥2/3 voting power signs.

Evidence types (slashable on UriChain)

- **Equivocation:** double-sign at same height/round.
- **Censorship/ Unavailability:** persistent downtime beyond threshold.
- **Cross-chain misbehavior:** fraud proven by a valid cross-chain proof.

Performance envelope

- Target block time on UriChain is ~1 s for global redundancy; U-Chains target ≤500 ms regionally. Governance can tune epoch length, dispute windows, and QoS tiers as empirical data accumulates.

1.2 Built-in DEX and Settlement

UriChain exposes an AMM like DEX (XYK + optional concentrated liquidity pools) as a system service. The DEX is used by: Universal Payments router to path-swap between URI and U-Chain local denominations and cross-chain asset egress/ingress for routing and settlement.

Invariant (XYK): For pool (X, Y), k = X·Y. Fees accrue to LP positions; a protocol fee may fund the Security Service reserve

1.3 Governance

A UriDAO controls parameters: epoch length, base fees, QoS tiers, oracle SLA classes, security auction curves, and registry policies.

2. Universal Cross-Chain Payments (UCP)

Any actor on any chain can pay any other actor across UriNet in **URI by default**, with optional local-currency acceptance when the receiver maintains on-chain liquidity and an auditable conversion path.

2.1 Canonical flow

1. **Intent formation.** The payer specifies: payer/payee, destination chain/address, amount, currency (URI or local), service class (base/priority), and optional memo. Fees are URI-denominated network-wide.

2. **Route resolution.** The Payments Router consults the **Chain Registry** to validate the destination, determine the canonical path, and confirm the payee's accepted currencies and service tier.

3. **Liquidity normalization.** If the payer's currency differs from URI or from the payee's accepted currency, the DEX Router executes the required swaps on UriChain (or approved ecosystem DEXs), respecting slippage bounds.

4. **Cross-chain settlement.** The **Messaging Hub** delivers a payment commit to the destination chain with verifiable headers and inclusion proofs. Upon validation, the destination credits the payee and returns an acknowledgement.

5. **Clearing, fees, and refunds.** Base fees and any QoS surcharges are charged in URI. Acks finalize the payment; timeouts unwind escrows and refund the payer.

2.2 Local-currency acceptance

Receivers may quote and accept their local token if they maintain sufficient in-pool liquidity and a secure on-chain conversion path to URI. Core settlement still clears in URI to keep pricing predictable.

2.3 Algorithm sketch — Payment routing & settlement (illustrative)

```
function pay(intent):
  assert(intent.amount > 0)
  route ← registry.canonicalRoute(intent.src, intent.dst,
intent.currency)
  quote ← dexRouter.quote(route, intent.amount, intent.maxSlippageBps)
  escrow.lock(intent.payer, intent.amount, intent.currency)
  swapped ← dexRouter.execute(route, intent.amount)
  pkt ← buildPaymentPacket(intent, swapped)
  icm.send(pkt)
  ack ← icm.awaitAck(pkt, intent.timeout)
  if ack.success: escrow.finalizeFees(); return SUCCESS
  else: escrow.refund(intent.payer); return TIMEOUT_OR_FAIL
```

2.4 Data Structures

```
message PaymentIntent {
    string intent_id;
    string src_chain_id;
    string dst_chain_id;
    string payer;
    string payee;
    string pay_denom;
    uint256 pay_amount;
    uint256 max_slippage_bps;
```

```
    uint64 timeout_height;
    uint64 timeout_time;
    bytes  memo;

}
 message PaymentAck {
    string intent_id;
    enum Status { SUCCESS=0; TIMEOUT=1; PARTIAL=2; FAIL=3; }
    Status status;
    string reason;              // nonempty on FAIL

}
```

2.5 Failure Handling

Timeout rolls back escrow and cancels path reservations. Partial only for multi-hop deliveries that partially fill within SLA; the remainder refunds.

3. Universal Cross-Chain Asset Movement (UAM)

UriNet standardizes voucher semantics and denom tracing to preserve fungibility across canonical routes. UAM borrows proven ideas (denom traces, acknowledgments, encryption on owner, chainid, etc.) and standardizes a Registry-anchored canonical path to avoid multi-channel fragmentation.

Scope. Movement of native U-Chain tokens and other digital assets across UriNet using standardized semantics and **registry-defined canonical routes**.

3.1 Mechanics

- **Canonical identity:** The Registry assigns asset identifiers (issuer/chain, denom, endpoints). This prevents routing ambiguity and fragmentation.
- **Transfer modes:** Lock-and-mint/burn-and-mint for representations; native transfer when the source chain supports it directly.

- **Fungibility protection:** Exactly one canonical route per (asset, source, destination) preserves a single fungible supply, avoiding multiple “wrapped” variants.
- **Encryption:** This enables instant, secure, authentic, and quick transactions.
- **Final outcomes:** Each transfer concludes with exactly one outcome—success, failure with reason, or timeout with refund—tracked by immutable acknowledgements.

3.2 Algorithm sketch — Canonical asset transfer

```
function transferAsset(req):
  route ← registry.canonicalRoute(req.src, req.dst, req.denom)
  ensure(balance(req.sender, req.denom) ≥ req.amount)
  escrow.increase(req.sender, req.denom, req.amount)
  pkt ← makeTokenPacket(req, route)
  pkt` ← Sign(pkt, sender,smart_contract_owner)
  icm.send(pkt`)
  ack ← icm.awaitAck(pkt`, req.timeout)
  if ack.success: escrow.decreaseFinal(); return SUCCESS
  else: escrow.releaseTo(req.sender); return TIMEOUT_OR_FAIL
```

3.3 Data Structures

```
message TokenTransferPacket { // analogous to ICS-20
   string src_chain_id;
   string dst_chain_id;
   string sender;
   string receiver;
   string denom; // base denom on source asset /... trace
   uint256 amount;
   bytes  memo;
   Signature signature;
}
```

```
message DenomTrace {
   // maintained at destination when voucher minted
   string base_denom; // e.g., uFOO
   string trace_path; // canonical channel path registry-resolved
}
```

3.4 Send/Receive State Machine (sender side)

```
 send_token(src, dst, packet):
    require(balance[src,sender,packet.denom] ≥ packet.amount)
    escrow[src,packet.denom] += packet.amount         // or burn for
native
 vouchers
    send_packet(AnyMsg{ type: TOKEN_TRANSFER, data: packet })
 on_ack(src, dst, ack):
    if ack.SUCCESS: decrease_escrow(src, packet.denom, packet.amount)
    else if ack.TIMEOUT: release_to_sender()
```

3.5 Receive/Mint State Machine (receiver side)

```
on_recv_token(dst, packet, proof):

   verify_proof(packet, proof)
   denom' ← compute_denom_trace(packet)
   mint(dst, packet.receiver, denom', packet.amount) write_ack(SUCCESS)
```

3.6 Canonicality and Fungibility

The Chain Registry designates a single canonical route per pair (ChainA, ChainB, Denom). If multiple channels exist, UAM normalizes to the registry's route to prevent fragmented denominations.

4. Interchain Communication Module (ICM)

[Especially under active development— subject to change] Any-message, RPC-like (gRPC) protocol for cross-chain calls, with ordered/unordered channels, QoS, encryption, and acknowledgments.

Message anatomy: Each packet includes: source/destination chain IDs, a channel ID and sequence, a payload type (e.g., payment, transfer, oracle, security), timeouts (height and/or time), and a compact commitment used for verification.

Channels & ordering: Ordered or unordered channels support different app needs; per-channel sequences and stored acks provide replay protection and exactly-once semantics.

Verification: BNS + Blockchain Router may verify the source before routing. Sender signature may be required.

Access control & QoS: Throughput is enforced by stake-unlocked tiers (base, staked, premium) and pay-as-you-go bursts, applied per chain/contract. Higher tiers secure higher quotas and priority scheduling; abuse triggers throttling and fines.

4.1 Algorithm sketch — ICM sender/receiver state machines

```
// sender side
send(packet):
  seq ← nextSeq(packet.channel)
  commit ← hash(header(seq) || packet.payload)
  store(commit); relay(packet, commit)

// receiver side
onRecv(packet, proof):
  verifyBNSRouter(packet.srcHeader)
  verifyInclusion(commit, proof)
  dispatch(packet.payloadType, packet.payload)
  writeAck(packet.channel, packet.sequence, success)
```

4.2 Packet and Channel Types

```
 message AnyMsgPacket {
    string src_chain_id; string dst_chain_id; string channel_id;
uint64 sequence;
    Signature signature;
    bytes payload;// opaque to transport; verified by dest module
    string payload_type;// CALL, TOKEN_TRANSFER, ORACLE, SECURITY,...
    uint64 timeout_height; uint64 timeout_time;
    bytes commitment;
 }
  message Ack { bool success; string error; bytes data; }
```

4.3 QoS and Access Control

Tiered bandwidth: Q0 (free/low), Q1 (staked), Q2 (premium). Each tier maps to rate limits (msgs/sec, bytes/day) and priority queues—designed around preventing DDOS and other misbehavior while simultaneously allowing for the accrual of fees for community maintenance, to service providers, validators, etc.

Admission: stake URI or pay per-byte to upgrade tier. Quotas enforced by per-channel token-bucket.

4.4 Transport and Security

Transport proofs: Merkle inclusion + light-client verification of the counterparty header.

Optional end-to-end encryption: payload encrypted with recipient service pubkey; headers remain clear for routing.

4.5 Sender/Receiver Algorithms

```
send_packet(packet):
 seq ← next_sequence(channel)
```

```
 commitment ← merkle_commit(packet_header(seq) || packet.payload)
store_commitment(seq, commitment)
 relay(packet)

on_recv(packet, proof):
 verify_light_client(dst→src header) verify_merkle_proof(commitment,
proof) route_to_module(packet.payload_type, packet.payload)
write_ack(true/false)
```

4.6 Timeouts and Acks

Timeouts return funds/state on the sender side. Acks are immutable receipts stored by both chains for replay-protection.

5. Security Service for Utility Chains (two complementary options)

[Notice] Projects can select one, compose both, or migrate over time.

Solution A — Validator Market (validators run nodes for the requester).

- **Request:** A U-Chain posts minimum committee size, duration, throughput target, SLOs, and a URI-denominated budget.
- **Matching:** UriNet validators bid capacity/price; the system selects a committee that meets fault-tolerance and throughput requirements at a clearing price.
- **Operations & enforcement:** Selected validators operate the U-Chain nodes under SLOs. Proven downtime, equivocation, or service violations are slashable on UriChain. Chains may eject/replace under-performers with orderly wind-down.

Solution B — Trust anchoring via higher-chain security (rollup-like).

- **Approach:** The U-Chain periodically posts checkpoints to a higher-security chain—UriChain or a dedicated Security Service Chain—at configurable assurance

levels: from root-hash checkpoints (low cost) to fraud-proofed or validity-proofed rollups (high assurance).

- **Service tiers:** Higher assurance implies higher fees; projects choose based on risk/cost. Posters are accountable; disputes and slashing follow standardized rules within defined windows.

5.1 Shared Concepts

Cross-chain slashing: Misbehavior on a secured U-Chain triggers slashing of the offender's stake on UriChain (or S-Chain), enforced by proof relay.

Proofs: double-sign evidence, fraud-proofs (optimistic), or validity proofs (ZK) depending on the selected mode.

Payments: all fees in URI; prices in URI-denominated units.

5.2 Solution A — Validator-Lease Security (VLS)

A marketplace where U-Chains lease a committee of UriChain validators to run validating nodes (full or light) for their chain.

Auction Inputs:

```
U-Chain request: (min_committee_num, duration_epochs,
throughput_target, SLA_downtime, budget_URI)
Validators bid: (throughput_offer, price_per_epoch, max_slots)
```

Allocation (uniform-clearing)

```
function allocate_committee(req, bids):
 sort bids by price_per_epoch asc
 pick smallest prefix achieving req.min_committee and
throughput_target       clearing_price ← last_selected.price_per_epoch
 committee ← { (v, clearing_price) for v in selected }
 return committee, clearing_price
```

Operation: Selected validators run the U-Chain nodes. Evidence of downtime/equivocation on U-Chain → cross-posted to UriChain → slash validators' URI stake proportionally to agreed SLA weights.

Rewards: Epoch reward per validator `R_v = clearing_price - penalties + QoS_bonus` . QoS bonus paid for meeting latency/throughput.

5.3 Solution B — Anchored Trust via Checkpointing (ATC)

U-Chains remain sovereign but periodically anchor commitments to a higher-security chain (Security Service Chain, or S-Chain).

Trust Tiers

- **T0 RootHash:** post block root periodically. Low cost; liveness attest only.
- **T1 Header+Signatures:** post header with validator sigs; enables equivocation proofs.
- **T2 tier:** using Solution A or alternative solutions such as a rollup.

Checkpoint Packet

```
message Checkpoint {
    string uchain_id; uint64 height; bytes state_root; bytes header;
bytes proof; Tier tier;
}
```

Posting Rule: U-Chain operators or designated posters submit Checkpoints to the S-Chain. S-Chain verifies according to tier and stores commitments for dispute windows.

5.4 Kick-Out and Reassignment

U-Chain may report misbehavior to the S-Chain, which may in turn eject misbehaving validators/committees mid-epoch. S-Chain runs a reassignment queue to refill slots.

5.5 Cross-Chain Slashing Evidence Flow

```
uchain.detect_misbehavior(e):
pack evidence (headers, signatures, witness)
send_packet(SECURITY_EVIDENCE, e) → S-Chain S-Chain.verify(e)
if valid: slash(staker_on_UriChain)
```

5.6 Payment and Accounting

Upfront reservation fee (nonrefundable) + per-epoch service fees

Usage fees: S-Chain gas fee + Security Tier fee

Escrow maintained on UriChain; shortfall triggers auto-pause of the secured service

5.7 Reserve and Backstop

A portion of protocol fees seeds a **Security Reserve** used to compensate innocent users in rare correlated faults (governance-controlled payouts).

6. Oracle Service (market + aggregation)

It provides curated external data feeds and attestations with explicit incentives for accuracy, timeliness, and availability.

Lifecycle:

- **Request & selection.** A chain posts a request (schema, latency/precision SLOs, and collateral policy). Oracles bid with price, expected latency, and reputation commitments; the coordinator selects a committee meeting SLOs at minimum cost.
- **Attestation & aggregation.** Providers submit signed reports (optionally via commit-reveal). Robust aggregation (median-of-n, stake-weighted filters, or similar) yields a final value.
- **Settlement & accountability.** Fees are paid in URI on acceptance. Missed SLOs trigger partial refunds; provable inaccuracies or equivocation within a dispute window result in

stake slashing and reputation penalties, with bounties for valid challenges.

6.1 Request and Bid

```
message OracleRequest {
   string req_id; string dst_chain_id; string query; string spec_uri;
// SLA,aggregator, min_n, quorum f
   uint64 min_responses; uint64 max_latency_ms; uint64
payment_per_report_uri;
   bytes collateral_policy; // slashing weights
  }

message OracleBid {
   string req_id; string oracle_id; uint64 price; string pubkey;
}
```

Allocation selects lowest-cost set meeting `min_responses` and latency.

6.2 Aggregation Phases

1) **Commit**: oracles sample off-chain sources; post `H(value||nonce)`.

2) **Reveal**: post `value,nonce`.

3) **Aggregate**: coordinator computes median (or VWAP) and forms a report signed by ≥q oracles; submit once on-chain.

4) **Dispute Window**: any party can submit counter-evidence; if successful → slash faulty reporters and pay bounties.

Pseudo

```
aggregate(req):
 C ← collect_commits(n)
```

```
R ← collect_reveals(n)
values ← select_valid(R, C)
y ← median(values)
sig ← threshold_sign(y) submit_report(req.req_id, y, sig)
```

6.3 Incentives

Payment per accepted report; **penalty** for missed reveal or outlier > k·MAD. Reporters post **collateral** on UriChain to be slashable across chains.

7. UriPortal (Ecosystem Infrastructure)

[Especially under active development—subject to change]

UriPortal is the open API gateway to UriNet: register/launch a chain, discover services, and access shared rails. **Staking URI** is required to join, mitigating Sybils and collateralizing penalties.

7.1 Chain Registry (on Portal)

Chain Registry. Canonical directory of chain ownership, base denom, endpoints, declared services, and canonical routes.

```
message ChainRecord {

 bool register, string chain_id; string owner; string endpoint_rpc;
string endpoint_icm; string base_denom; repeated string
supported_denoms;
 repeated string declared_services; // e.g., PAYMENTS, ORACLE, KYC,
MESSAGING string canonical_channel_map; // JSON mapping of (peer,
denom) → channel

   string version; uint64 registered_at; bool active;
}
```

Service Registry. Canonical directory of chain ownership, base denom, endpoints, declared services, and canonical routes.

```
message ChainRecord {
 bool register, string chain_id;  string service_id; string owner;
string endpoint_rpc; string endpoint_icm; string base_denom; repeated
string supported_denoms;
string declared_service; // e.g., PAYMENTS, ORACLE, KYC, MESSAGING
string features;
repeated string apis;
string canonical_channel_map; // JSON mapping of (peer, denom) →
channel
string version; uint64 registered_at; bool active;
}
```

Functions

```
register_chain(record): require(stakeURI ≥ minStake); store(record)
update_channel_map(chain_id, map): onlyOwner
resolve_route(chainA, chainB, denom): return canonical channel, hops
```

7.2 Services API (selected)

Stable interfaces/schemas for payments, messaging, oracle requests, and security-as-a-service, with registration, versioning, and discovery.

`POST /payments/intents` → creates PaymentIntent and returns routing.

`POST /icm/send` → submits AnyMsgPacket.

`POST /oracle/request` → opens a request auction.

`POST /security/lease` → opens a VLS auction.

`GET /registry/chain/{id}` → fetch ChainRecord.

7.3 Discovery & Analytics

Search and metrics (latency, fees, uptime, usage, tokenomics) to help builders select providers and routes.

It also provides information such as index availability, latency, fees, TVL, etc.. It provides SDKs for ease of development and service access.

7.4 Joining and Staking

New chains must stake a minimum URI; slashing for spam/abuse (e.g., repeated timeouts, invalid packets) and refunds for honest de-registration.

8. Performance Targets and Engineering Plan

Targets (v0.1 testnet → v1.0 mainnet): Block time: ≤ 500 ms per U-Chain (regional validator topology); UriChain at ~1.0 s for higher geo-redundancy. Throughput: ≥ 10,000 TPS/chain sustained for simple value transfers (200–250 B/tx), p95 end-to-end payment latency ≤ 2.0 s (includes routing, swap, interchain ack). Finality: < 2 blocks subjective (fast optimistic), ≤ 2–3 blocks economic (≥ 2/3 commits observed).

Capacity Planning (order-of-magnitude): If `avg_tx_size = 220 B` and `TPS = 10k` , then payload throughput ≈ 2.2 MB/s. With T_block = 0.5 s , per-block payload ≈ 1.1 MB. Add consensus and receipts overhead (~0.4–0.7 MB) ⇒ block size target ≈ 1.5–1.8 MB. Gossip fanout f = 6 , overhead ≈ ln(n) stages; with BLS aggregate signatures, commit payload ≈ 48 B + n/8 B (bitmap) vs n·64 B without aggregation.

Consensus micro-timers (indicative for U-Chains): `propose ≤ 180 ms , prevote ≤ 150 ms , precommit ≤ 120 ms , commit ≤ 50 ms ⇒ ≤ 500 ms` end-to-end, leveraging pipelined BFT and aggregate signatures. Regional peering and relay accelerators (UriPortal edge relayers) maintain `RTT_95 ≤ 120 m`s.

Execution Budget/Weights: Define a per-block weight W_block and per-tx weight w_tx ; admission uses a token-bucket so Σ w_tx ≤ W_block . Default W_block sized to admit 5,000–6,000 simple tx per block at 0.5 s (headroom for ICM/Oracle/DEX ops).

Storage/State I/O: Merklized accounts (key/value) with batched state writes per block; write-amplification kept < 1.6× via WAL + batch. - Snapshots every `K = 2,400` blocks (~20 min at 0.5 s) for fast bootstraps/ light clients.

Economics

At a Glance

URI token is the standard unit of utility across UriNet—powering access, staking/governance, payments, and participation in/for all network services and shared modules.

- **Genesis supply:** 1,000,000,000 URI.
- **Denominations:** 1 URI = 10^9 Kwak (KWAK), the minimum sub-denomination (10^-9).
- **Inflation bounds:** Chain-configurable floor and ceiling with initial parameters set to a **minimum of 1%** and a **maximum of 10%** per year.
- **Transaction fees (base + priority) and burning:** Every transaction pays a **base fee** (per signature). **50% of the base fee is burned** and **50% goes to the current block leader (proposer).** Users can optionally add **priority fees** (aka compute-unit price) to get included faster. **100% of priority fees go to the block leader**.
- **Automatic control (default):** Inflation adjusts each block toward a **target staking ratio of 67%**. If stake is below target, inflation moves up to incentivize staking; if above, it moves down.
- **Manual override (emergency/exceptional):** Governance can set inflation directly when warranted, subject to on-chain quorum and supermajority (see Manual Override).
- **Reward routing:** A dedicated minting module mints block rewards and hands them to the distributor module, which deducts any community tax and proposer reward (if any), then pays validators pro-rata by voting power; delegators receive rewards net of validator commission.

- **Governance agility:** Monetary parameters are module settings; approved changes take effect immediately after the voting period and are applied starting with the very next block.

Token Utility

- **Security:** Stake/collateralize URI to participate (directly or via delegation) in UriChain consensus and various services.
- **Fees:** URI pays for ecosystem services, execution, messaging, and settlement across UriNet (mainnet and Utility Chains).
- **Incentives:** Validators and delegators earn URI emissions and fee-based rewards; proposers receive a small, chain-configurable bonus.

Inflation Policy

1) Automatic Inflation Adjustment (default)

Core parameters (governance-set):

- `goal_staked`: target staking ratio; initially 0.67 (two-thirds).
- `inflation_min`, `inflation_max`: hard bounds; initially 1% and 10% annualized.
- `inflation_rate_change`: maximum annual rate at which inflation can move toward the target (applied incrementally per block).
- `blocks_per_year`: cadence used for pro-rata minting; with 0.5s blocks, this is 2 × 86,400 × 365 = 63,072,000.
- `mint_denom`: base denomination for minting and accounting (KWAK).

Per-block mechanism:

1. **Measure staking:**
   `staked_ratio = staked / total_supply`

- **Update inflation (bounded):**
  `inflation_rate_change_per_year = (1 - staked_ratio / goal_staked) * inflation_rate_change`
  `inflation_next = clamp(inflation_current + inflation_rate_change_per_year / blocks_per_year, inflation_min, inflation_max)`

2. **Intuition:** if `staked_ratio` < `goal_staked`, the term is positive and inflation rises; if above, it falls.

3. **Compute provisions:**
   `annual_provisions = inflation_next * total_supply`
   `block_mint = annual_provisions / blocks_per_year`

4. **Mint & route:**
   The mint module creates `block_mint` (in KWAK) and sends it to the fee collector.

5. **Distribute at block start:**

Example (illustrative):
Assume `goal_staked = 0.67`, `inflation_rate_change = 0.10` (10%/yr slope), `inflation_current = 0.035` (3.5%), and `staked_ratio = 0.50`.
Then `1 - 0.50 / 0.67 ≈ 0.254`; per-year change ≈ 0.0254. Per-block delta is ≈ 0.0254 / 63,072,000 ≈ 4.03×10^-10. Inflation ticks up by that amount this block (bounded by 1–10%). This slow, continuous controller dampens volatility and makes the security budget adaptive without governance micromanagement.

2) Manual Override (exceptional governance)

When urgent conditions or clear public-interest reasons exist (e.g., acute security budget gaps, systemic shocks), governance can **temporarily set inflation and/or community tax rate (if any) directly** or adjust parameters outside the normal cadence.

- **Quorum:** At least **50% of staked weight** participates in the vote.
- **Supermajority:** Among participants, at least **two-thirds (⅔)** must vote to approve.
- **Effectivity:** Upon approval, the new rate/parameters replace the old ones and take effect **immediately after the voting period**; the **very next block** uses them.
- **Scope:** Changes may set a fixed inflation for a defined period or update controller parameters (`goal_staked`, bounds, slope).
- **Reversion:** Unless otherwise specified, the system reverts to the automatic controller after the override window elapses.
- **Implementation detail:** modules expose an authority address (typically the governance module/DAO). A successful parameter-change proposal updates the live parameters atomically.

Emissions and Fee Distribution

Newly minted tokens and collected fees are routed as follows:

1. **Community tax (optional):** If enabled, a small percentage of each block's rewards/fees is diverted to the **community pool** for grants, maintenance, and ecosystem programs (rate and uses are governance-configured).
2. **Proposer reward: 0% (default, block reward). However,** DAO can vote to set **1–5%** proposer bonus (chain-configurable) to incentivize low-latency, high-availability block production.
3. **Security rewards:** The remainder is distributed **pro-rata by voting power** across **staked validators**; delegators receive their share net of the validator's commission.
4. **Transaction fees (base + priority) and burning:** Every transaction pays a **base fee** (per signature). **50% of the base fee is burned** and **50% goes to the current block leader (proposer)**. Users can optionally add **priority fees** (aka compute-unit price) to get included faster. **100% of priority fees go to the block leader**.

Rewards accrue continuously and can be claimed by recipients on demand.

Governance and Parameter Management

Inflation behavior is entirely **parameterized**. Governance can update:

- Controller target and slope: `goal_staked`, `inflation_rate_change`
- Bounds: `inflation_min`, `inflation_max`
- Cadence and denom: `blocks_per_year`, `mint_denom`
- Distribution knobs: community tax rate, proposer reward rate, transaction fee burning ratio, etc.

A successful parameter-change proposal updates the active configuration; changes **apply beginning with the next block** following the voting period. This gives UriNet both **predictability** (automatic control) and **optionality** (fast-acting override) without protocol upgrades.

Supply, Units, and Accounting

- **Genesis supply:** 1,000,000,000 URI allocated per the genesis distribution plan.
- **Units:** 1 URI = 1,000,000,000 KWAK; all minting and settlement occur in KWAK to avoid rounding.
- **Total supply accounting:** Updated each block as emissions are minted and burned (if/when fee burns or clawbacks are introduced by governance in the future).

Design Rationale

- **Security-first:** A target staked ratio of ~⅔ balances decentralization, economic security, and liquidity for real-world utility.

- **Predictable yet responsive:** Per-block, bounded adjustments avoid fee and reward whiplash while still reacting to sustained shifts in stake.
- **Clear roles and incentives:** Proposer bonuses and optional community tax ensure operational excellence and long-term commons funding without diluting validator/delegator alignment.
- **Governance agility:** Immediate, block-level effectivity of approved parameter changes enables rapid crisis response without hard forks.

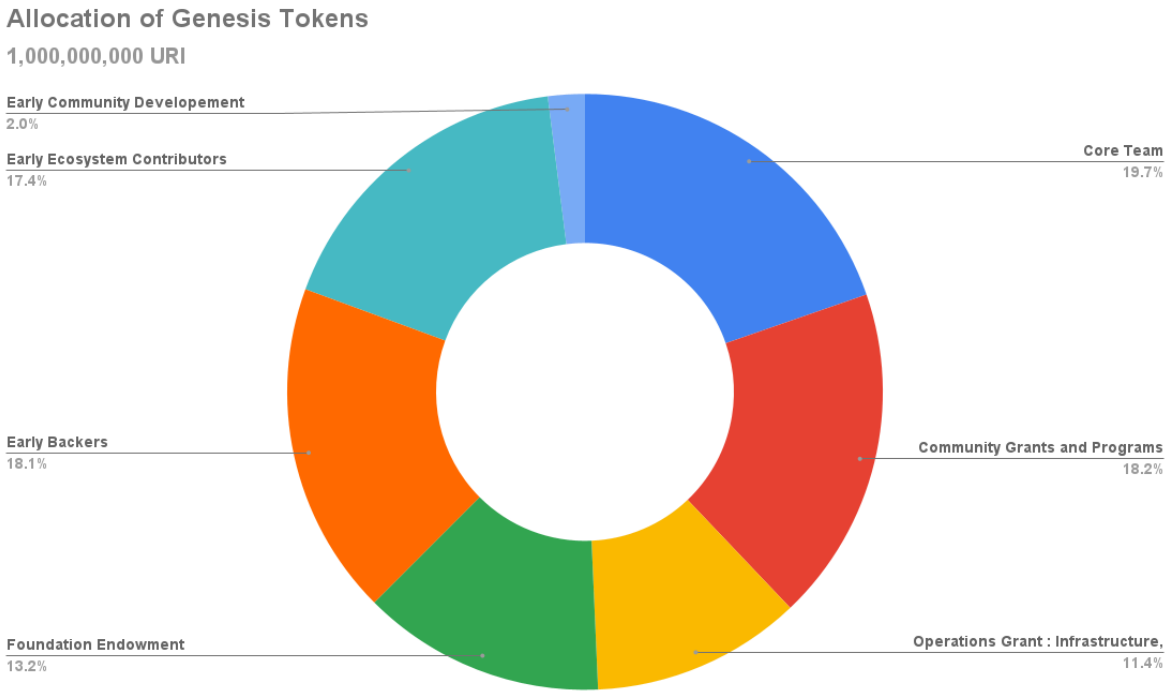
At-a-Glance Parameters (initial, governance-tunable)

- `goal_staked`: 0.67
- `inflation_min`: 0.01
- `inflation_max`: 0.10
- `inflation_rate_change`: (set by governance; defines the controller's annual slope)
- `blocks_per_year`: 63,072,000 (0.5s blocks)
- `proposer_reward`: 0%–5% of block rewards/fees (0% default)
- `Fee_burting_ratio: 0.5 (`50% of the base fee is burned)
- `community_tax`: 0–X% (optional; to the community pool)
- `mint_denom`: KWAK (10^-9 URI)

By combining a **self-stabilizing controller** with **clear distribution mechanics** and **fast governance levers**, UriNet's economics prioritize durable security, predictable developer/user experience, and the long-term health of the ecosystem.

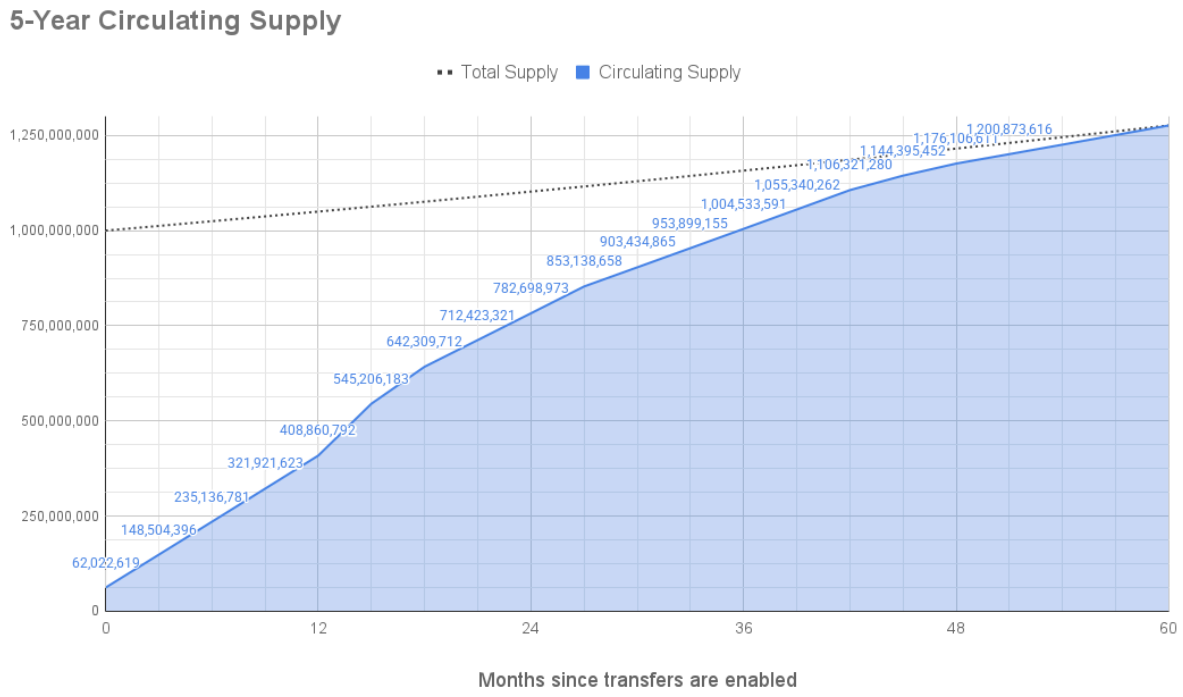
Overall Allocation and Supply Schedule

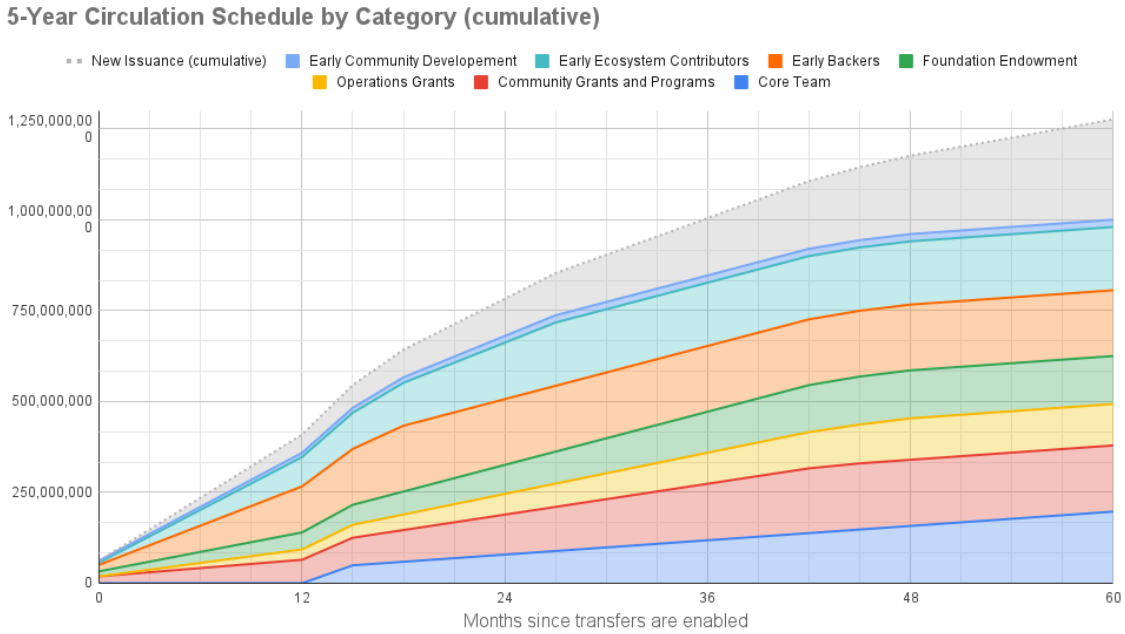
There will be 1,000,000,000 URI tokens created at Genesis. It is planned to be allocated in the following way, where each category is described in greater detail in the following sections:



“Circulating Supply” is every token that isn’t subject to a lockup at a particular time. The vast majority of the initial supply of URI tokens is subject to long-term lockups.

The chart below shows the increase in circulating supply as tokens unlock and new tokens are minted. The dotted line shows the total supply as it increases due to new issuance (but ignores any burning that would be due to fees). This and all charts WILL begin on the date that transfer restrictions are lifted by Governance/DAO/Foundation.

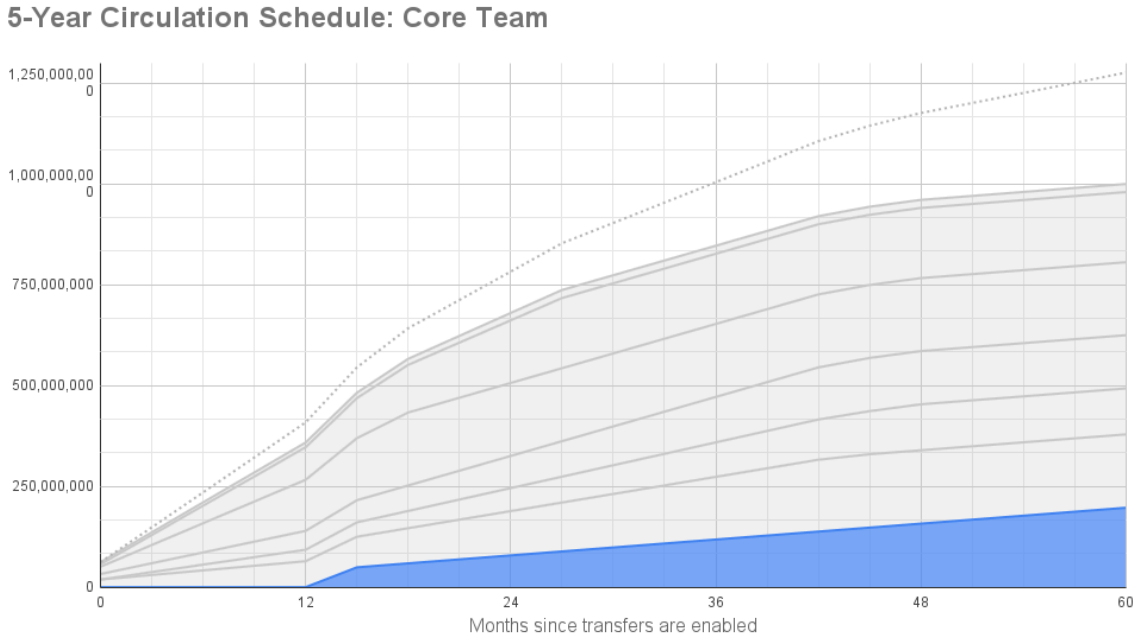




Core Team

This allocation recognizes the people who turned the idea into running code. It covers founding engineers, product and protocol researchers, security leads, and key advisors whose work

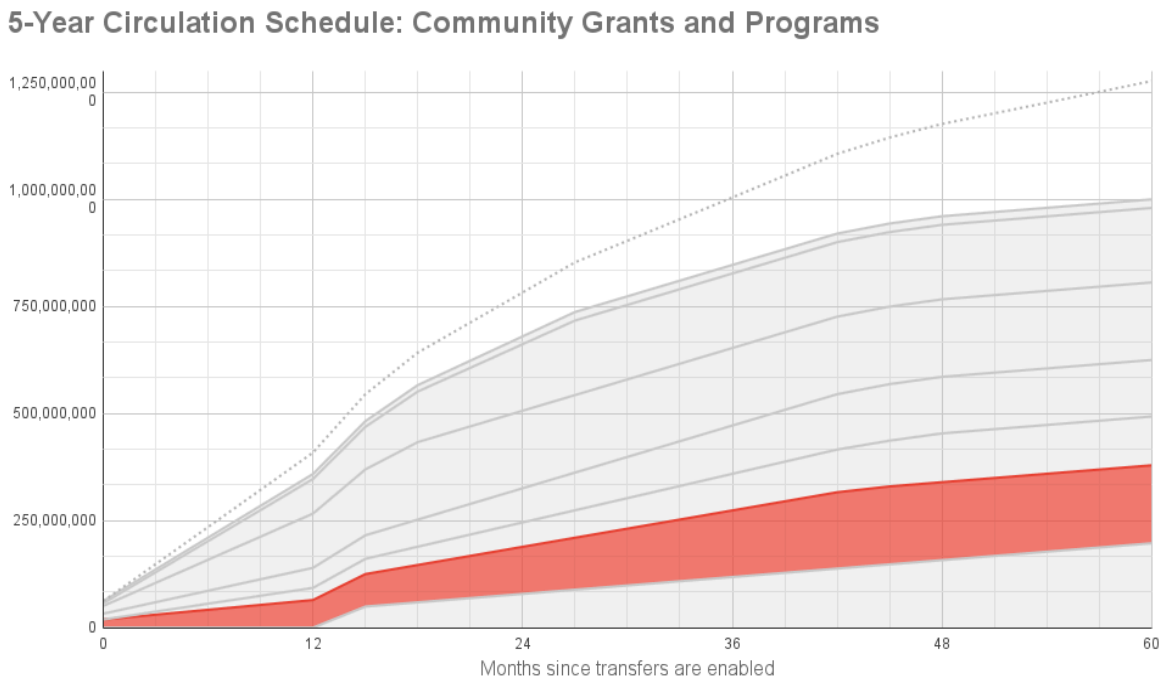
spans multiple releases. Tokens here are long-term oriented: subject to multi-year vesting with an initial cliff, aligning incentives with delivery of roadmap milestones rather than short-term market moves. A portion is reserved for future strategic hires so we can recruit top talent without repeatedly tapping the treasury. Day-one liquidity is intentionally limited to keep this bucket aligned with network health and long-run value creation.



Community Grants and Programs

This pool funds public-goods style work that grows the pie for everyone: protocol research, developer tooling, libraries and SDKs, documentation, education, hackathons, and community-run initiatives. Grants are milestone-based and evaluated by an open process with clear criteria; larger, multi-quarter efforts may be staged with tranches and progress reviews. We

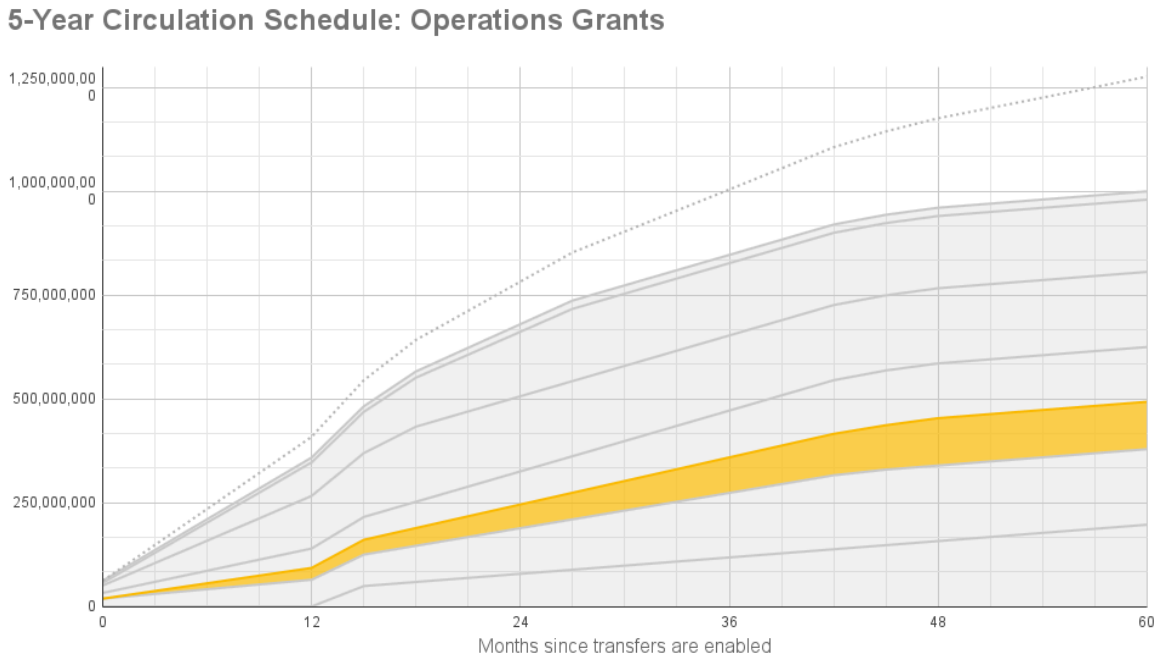
expect most outputs to be open-source and permissively licensed so they’re reusable across the ecosystem. Distribution is paced over time to avoid shocks and to ensure we can support promising new work as real-world needs emerge.



Operations Grant: Infrastructure, Operation cost, etc...

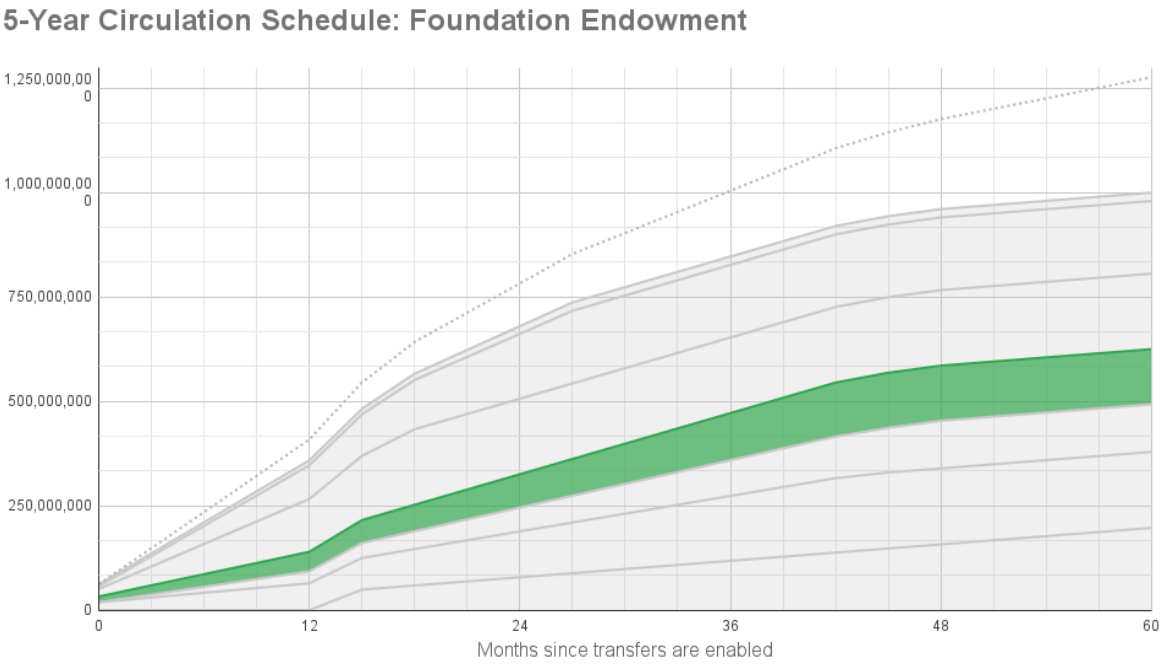
Launching and running a network takes reliable, boring excellence. This bucket covers essential operating expenses that don’t fit cleanly into “grants” or “team” compensation: security audits and bug bounties, observability and data infra, seed validator bootstrapping or testnet incentives, wallets/explorer maintenance, developer support, legal/compliance and statutory filings, insurance, incident response, and vendor costs (hosting, CI/CD, etc.). Where practical,

payouts are denominated in stable assets or scheduled to minimize market impact. The goal is predictable, transparent funding of the “keep-the-lights-on” work that underwrites user trust.



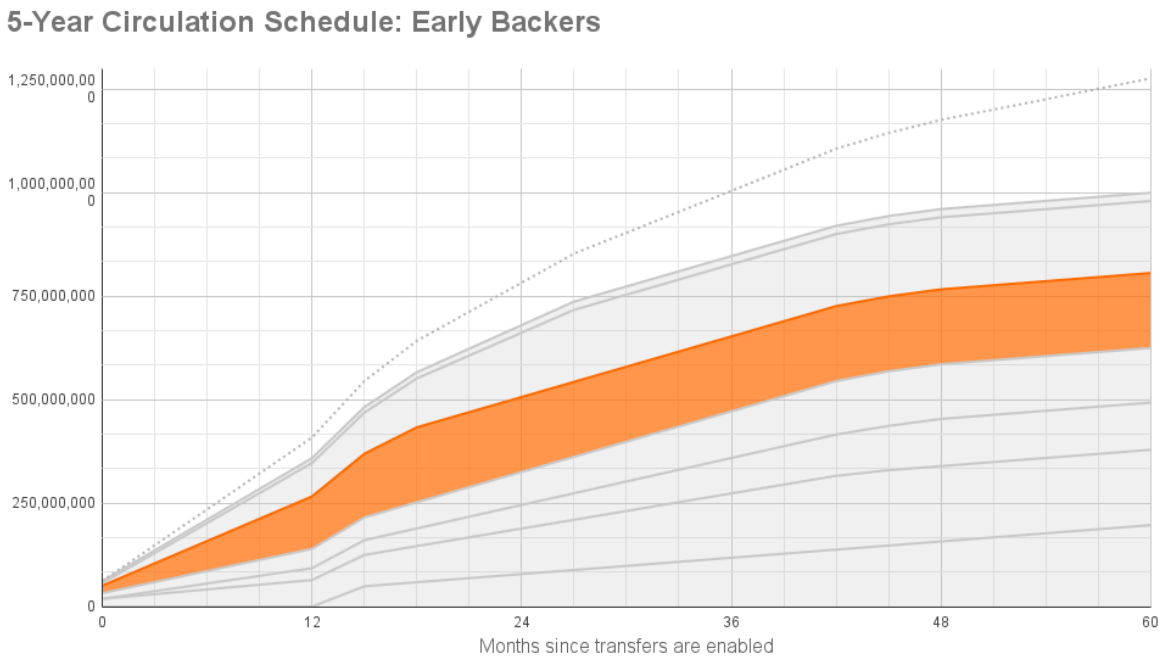
Foundation Endowment

The endowment is the long-horizon backbone of ecosystem stewardship. It sits with the independent Foundation and is governed by a published spending policy that favors stability over cycles. Proceeds are used to fund non-commercial public goods, core protocol initiatives, academic/standards collaborations, etc.. Custody, reporting, and conflict-of-interest rules are conservative by design, with periodic transparency reports on holdings, performance, and disbursements. This helps ensure the project can serve builders and users for years, not just a market season.



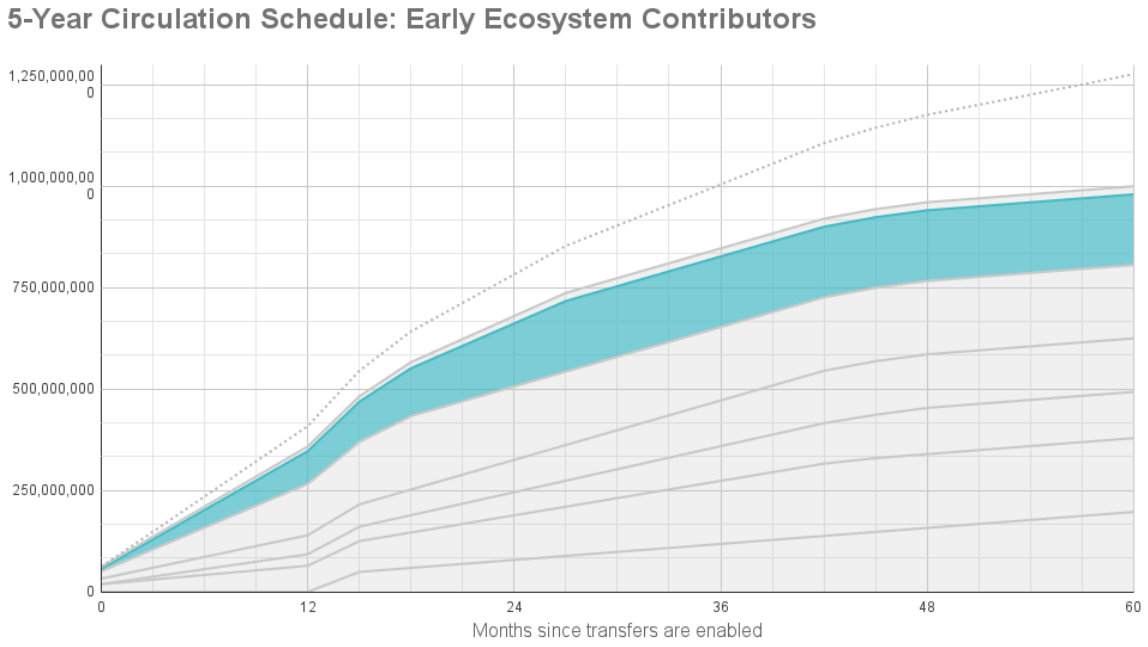
Early Backers

Early backers supplied the capital that made multi-year R&D and early ecosystem formation possible before there was usage or revenue. Tokens are subject to standard lockups (for example, cliffs followed by linear vesting), transfer restrictions during sensitive phases, and clear disclosures.



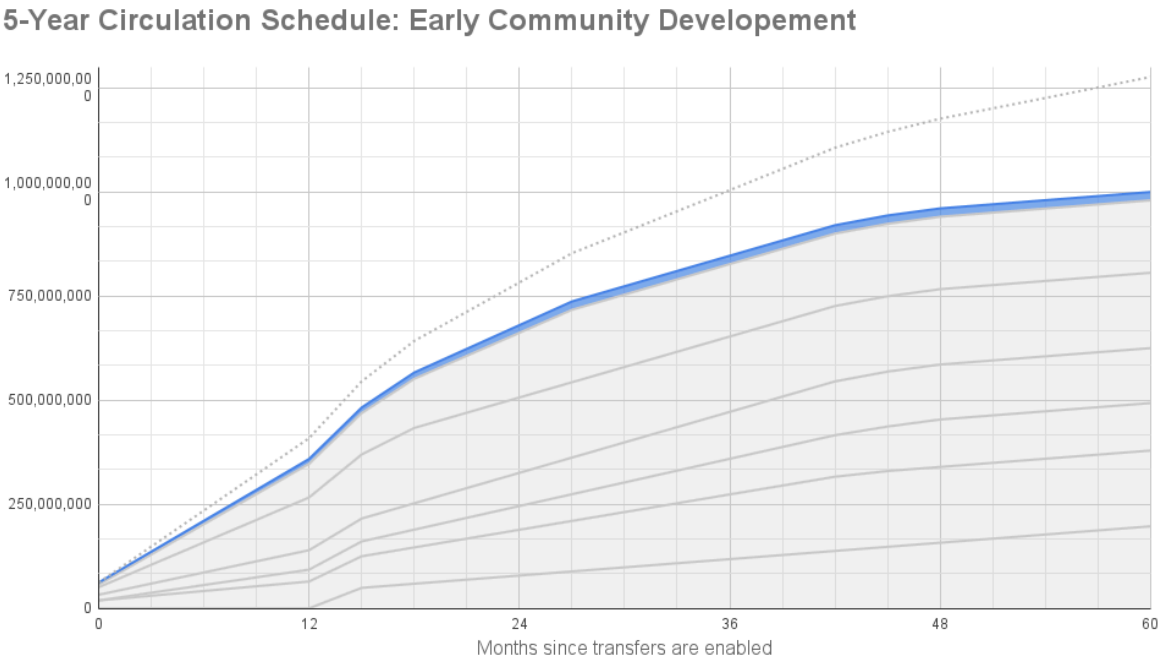
Early Ecosystem Contributors

This pool rewards the folks who show up early and ship. It includes teams that launched foundational applications or utility services (payments, identity/compliance, oracles, data, comms), operators who supported incentivized testnets or critical community tools, and partners who onboarded real users or enterprises. Distributions are milestone-based and may include performance-linked vesting (for example, tied to uptime, usage thresholds, or open-sourcing commitments). The aim is to recognize concrete contributions that made the network useful from day one and to keep these builders invested in its growth.



Early Community Development

This pool jump-starts the first wave of real users and contributors. It covers things such as airdrops, newcomer onboarding, simple quests, local meetups, translations, and community-run tools/moderation. The goal is straightforward: seed healthy network effects and give early ownership to the people who show up and stick around.



Real-World Use Cases

UriNet is designed for everyday utility: each product runs on its own sovereign Utility Chain, natively connected to others through UriChain and UriPortal. Below we outline concrete use cases that map directly to the modules and interchain messaging described earlier, consolidating every item in our internal notes into coherent, no-fluff application categories.

Example Applications (subject to applicable regulations)

1) Finance & Digital Markets

RWA/STO issuance and servicing. Launch compliant, single-purpose chains for asset tokenization (real estate, revenue-share notes, invoices), with on-chain cap tables, distributions, and transfer restrictions. Identity/compliance modules gate access; universal payments disburse coupons/dividends.

NFTs and digital goods. Per-product chains mint and trade NFTs tied to utility (tickets, memberships, game assets). Native interchain messaging lets assets move between games/apps.

Exchange and settlement. UriNet's built-in DEX plus native messaging enables orderbooks, RFQ, or AMM models across chains—settlement occurs atomically network-wide without brittle bridges.

Payments & benefits. Issue programmable benefits, rebates, and loyalty points; settle across chains with universal payments.

Insurance & hedging. Parametric products trigger from oracle proofs (weather, shipping events, price feeds). Users can hedge risks or take positions via a network-wide prediction/derivatives chain.

Research & prediction. A dedicated prediction-market chain offers polls, forecasting, and hedging instruments; results can inform payouts or governance on other chains.

2) Trustless Private Contracts & Escrow

Spin up escrow/trust chains for bilateral or multiparty deals (marketplaces, B2B trade, creator collaborations). Templates include milestone escrow, claim-based insurance, and censorship-resistant agreements enforced by consensus and oracle proofs.

Oracle-triggered execution. Contracts finalize or release funds based on real-world facts supplied by UriNet's oracle chains, removing manual reconciliation and disputes.

3) Social, Media, and Advertising

User-owned social graphs. Community chains manage identity, social relationships, moderation policies, and content storage pointers. Users actually own their handles/content; portability across apps comes by default.

Ads monetization (on/offline). Dedicated ad chains clear impressions, clicks, and conversions with verifiable oracles. Revenue splits execute automatically among creators, publishers, and advertisers; works for online and offline activations (e.g., QR-in-store).

News networks: triple-win model. A news-vertical chain coordinates readers, advertisers, and publishers via transparent metrics, pay-per-article micro-payments, and ad revenue sharing—aligning incentives and reducing fraud.

4) Philanthropy, Public Goods, and Civic Tech

Transparent donations. Charity chains provide end-to-end traceability from donor to program to outcome; programmatic disbursements tie to oracle-verified milestones.

Public-sector tooling. Municipal or agency chains publish budgets, RFPs, and spend in near-real time; citizens audit flows and outcomes. Supports private, trusted deployments where sensitive data isolation is required.

5) Gaming and Interactive Apps

Game-specific chains. Each title (or studio) gets its own app chain for low-latency gameplay, item minting, tournament logic, and marketplace rules—interoperable by default with other titles. Prediction and poll features can be composed for esports formats.

6) Oracles and Real-World Data

Oracle utility chains. UriNet includes native oracle chains that feed prices, events, attestations, and IoT signals to all other chains. Contracts subscribe to these feeds, enabling autonomous execution in finance, insurance, supply chain, and civic apps.

Oracle auction. A marketplace chain lets apps request data with explicit SLAs; oracle operators bid to fulfill requests, staking collateral against accuracy and timeliness. This commoditizes reliable data and reduces costs network-wide.

7) Metaverse + AR/VR

AR/VR-first chains anchor identity, assets, and economic rules for virtual spaces that interact with the physical world. Ads, news, prediction games, and insurance can plug in as composable services within immersive environments.

How builders assemble these

Across all categories, builders compose a few recurring patterns:

- **Escrow + Oracle Trigger:** Funds locked on a contract chain; release conditions verified by an oracle chain (e.g., delivery scanned, rainfall threshold met). Useful for trade finance, parametric insurance, and service marketplaces.
- **Ad Impression Clearing:** Ad chain logs signed impressions; oracle verifies conversions; payout contract splits among stakeholders (creator/publisher/advertiser) with on/offline support.
- **Prediction ↔ Settlement Loop:** Prediction chain prices future states; finance/insurance chains read outcomes to settle hedges or unlock bonuses.
- **Donation Traceability:** Donor pays into a charity chain; milestones verified by oracle; disbursements and program spend are publicly queryable; civic dashboards reflect government-level tracking when used by public agencies.
- **Game Asset Interop:** Game chains mint items/NFTs; marketplace/DEX chains provide liquidity; identity/social chains carry achievements and access across titles.

Why UriNet fits these use cases

- **Sovereign, single-purpose chains** keep latency low and policy surfaces small, while **native interchain messaging** moves assets and actions without brittle bridging.
- **Shared core services**—universal payments, built-in security, identity/compliance, data/oracles, discovery, and a built-in DEX—are provided once and reused everywhere through **UriPortal**, giving web-simple UX with rigorous assurances.

Governance

Governance in UriNet spans four areas:

1. how the core protocol changes,
2. how shared resources are allocated,
3. how the network's **Utility Chains Manager** (Foundation-run essential/service chains and official support) is overseen
4. how **UriPortal** (the ecosystem gateway, registry, and APIs) is specified and updated.

UriChain (the mainnet) provides the on-chain governance rails and treasury; sovereign Utility Chains govern themselves locally but agree to baseline interop rules to access shared services through UriPortal.

What gets governed where

- **UriChain (network level).** On-chain votes decide protocol upgrades, network parameters, treasury allocations, and UriPortal/Registry/Services-API changes. UriChain coordinates settlement, payments, and the built-in DEX.

- **Utility Chains (local).** Each Utility Chain sets its own application logic, tokenomics, and validator policy. Network governance never rewrites a Utility Chain's state; it governs ecosystem privileges: listing in the Chain Registry, DEX routing, oracle/security access, and compatibility labels exposed via UriPortal.

- **Utility Chains Manager (UCM).** Initially, the Foundation operates community-critical essential/service chains (e.g., security, oracle, messaging, social) and coordinates their upgrades, support, and grants. Over time, mandates, budgets, and upgrades for these chains are set by network votes.

- **UriPortal governance.** The community votes on UriPortal's protocol, parameters, and APIs: Chain Registry policy, Services-API versioning, discovery rules, and deprecation schedules—so interop remains predictable as UriNet grows.

How decisions happen

- **Discuss off-chain, decide on-chain.** Ideas are debated in public forums, then submitted as proposals with clear, executable payloads. Validators, delegators, and registered participants vote on chain; deposits deter spam.

- **Simple proposal classes.**

    1. Core protocol upgrades and parameter changes on UriChain
    2. Treasury budgets and time-bound grants
    3. Registry/standards and UriPortal API changes
    4. UCM mandates and essential/service-chain upgrade coordination

- **Upgrade rollout.** Each approved protocol release passes through public testing and canary activation, then schedules mainnet activation at a specific height/epoch once readiness signals are met.

Resource governance (Treasury)

Network revenues (e.g., URI-denominated network/messaging fees, a share of DEX fees, join deposits, and penalties) fund the commons:

- **Essential/Service Chains:** baseline funding to keep shared services reliable

- **Grants & R&D:** milestone-based builder grants with clawbacks for non-delivery

- **Operations & Infra:** audits, monitoring, incident response, tooling

- **UriPortal Maintenance:** Registry operations, API evolution, documentation
  Disbursements are transparent and streamed/vested; progress reviews can pause or revoke streams.

Safety & emergencies

For rare, high-risk incidents (critical bugs, cross-chain failures), governance can authorize a narrowly scoped, time-limited emergency action (e.g., temporary quarantine of a misbehaving chain or pause of a vulnerable parameter). Every action includes a short challenge window, automatic expiry unless ratified, and a mandatory post-mortem.

Bootstrap → maturation

At launch, the Foundation coordinates UCM operations and maintains UriPortal's open-source rails. As participation scales, authority migrates fully on-chain: the community elects or renews maintainers, sets budgets, ratifies standards, and directs the UCM within defined guardrails.

Why this fits UriNet

The four scopes map directly to UriNet's primitives: a network-level DAO on UriChain for shared decisions and funding, sovereign Utility Chains for product-level control, a Foundation-run UCM to ensure reliable shared services from day one, and UriPortal's Registry/Services-API as the enforceable interface for interoperability. The result is simple, predictable governance that keeps UriNet evolving without compromising sovereignty or safety.

Roadmap

Phase 0 — Backbones & Presence

Establish public presence, legal/foundation scaffolding, and early community/partner funnels.

Key Deliverables:

- Whitepaper 0.1 and public homepage, plus initial social/community channels.
- UriNet Foundation setup and initial governance docs.
- Early Partners Initiative across content and RWA/STO funnels: news/publishers, ad networks, influencer agencies, online/offline ad firms, and RWA/STO companies.

Phase 1 — Public Testnets & Core Modules

Prove core networking, consensus, and developer loop; ship first-useful modules in a public testnet.

Key Deliverables:

- Testnet v1 with modified BFT-DPoS (with PoA option for devnets), EVM smart-contract support.
- Core modules: inter-chain communication (native messaging), Universal Payments, social/community primitives.
- Built-in DEX (testnet) with initial LP rules; reference Wallet and Block Explorer.

Phase 2 — Mainnet v1

Launch UriChain mainnet and deliver production-grade base functionality.

Key Deliverables:

- Mainnet v1 with finalized consensus configuration (modified BFT-DPoS).
- Built-in DEX v1 (mainnet) and LP rules; Interchain Asset Transfer (native).
- Smart-contract support: EVM at launch; Rust-based Smart Contract Compatibility to follow

Phase 3 — Foundation/Community-Managed Utility Chains

Enable official and community utility chains for real-world use, with packaged services.

Key Deliverables:

- Utility Chain Yellow Paper and SDK; “Run” tooling and templates to launch domain-specific chains.
- Oracle Service chain; Security Service Chain upgrade; RWA/STO template chain; private-chain profile.
- Built-in DEX upgrade path tied to variable/parameter governance across chains.

Phase 4 — UriPortal & Open Standards (Gateway)

Make UriPortal the ecosystem gateway and standardize how services are discovered, consumed, and extended.

Key Deliverables:

- Launch UriPortal services (discovery, APIs, payments, identity/compliance surfaces).
- Publish standards/protocols for launching third-party Portal services; Open API specs and developer guides.
- SDK for ecosystem infrastructure (Portal-facing).

NOTICE AND DISCLAIMER

PLEASE READ THIS ENTIRE NOTICE AND DISCLAIMER CAREFULLY. NOTHING HEREIN CONSTITUTES LEGAL, FINANCIAL, BUSINESS, ACCOUNTING, OR TAX ADVICE. YOU SHOULD CONSULT YOUR OWN INDEPENDENT LEGAL, FINANCIAL, TAX, OR OTHER PROFESSIONAL ADVISORS BEFORE ENGAGING IN ANY ACTIVITY RELATED TO THIS DOCUMENT, THE URI TOKEN, OR THE URINET NETWORK.

Neither the project's sponsoring entity or entities (together, the Organization), any developers, core contributors, advisors, contractors, or service providers (together, the Team), nor any distributor or vendor of URI (each, a Distributor) shall be liable for any direct or indirect losses, damages, or claims arising from or related to this Whitepaper, any related websites, repositories, or materials (collectively, the Materials).

Project purpose

The project seeks to research, design, build, and promote a utility-focused blockchain ecosystem (UriNet) intended to support real-world applications and improve everyday life. UriNet's native token, URI, is intended solely as a functional, cryptographically secured utility token used within the UriNet ecosystem. The Organization may support research, development, community programs, and operations that further this purpose.

Nature of this Whitepaper

This Whitepaper is provided strictly for general informational and discussion purposes only. It is not a prospectus, offer document, advertisement, or solicitation for investment, and does not constitute an offer to sell, a solicitation to buy, or a recommendation regarding any securities, financial instruments, or other regulated products in any jurisdiction. The Materials do not and are not intended to create any contractual or other legal rights between you and the Organization, the Team, any Distributor, or any affiliate thereof.

The information herein may be incomplete, inaccurate, or outdated. Assumptions, architectural descriptions, token functionality, roadmaps, timelines, and other content are preliminary, illustrative, and subject to change, suspension, delay, or abandonment at any time, at the Organization's sole discretion, without notice. No representation, warranty, or undertaking (express or implied) is made as to the accuracy, completeness, reliability, merchantability, fitness for a particular purpose, non-infringement, or timeliness of the Materials.

If the Materials contain information sourced from third parties, the Organization and the Team have not independently verified such information and make no representation or warranty regarding it.

No offer; definitive terms control

Nothing in the Materials constitutes, or should be construed as, an offer to sell or the solicitation of an offer to buy URI or any other asset. Any sale, distribution, exchange, or transfer of URI (if any) will occur, if at all, only pursuant to definitive documentation governing such transaction (e.g., token sale terms, token purchase agreement, acceptable use policy, privacy policy, or network terms, collectively, the Definitive Terms). The Definitive Terms, if and when provided, will control in the event of any inconsistency with this Whitepaper. No person is obligated to enter into any contract or binding legal commitment relating to URI based on the Materials.

Eligibility; compliance; sanctions

Access to URI or participation in UriNet may be restricted or prohibited for certain persons or in certain jurisdictions. Without limiting the foregoing, you represent and warrant that: (a) you will ensure at your own expense full compliance with all laws, regulations, and restrictions

applicable to you; (b) you are not a citizen, resident (tax or otherwise), domiciliary, or green-card holder of, located in, or otherwise ordinarily resident in any jurisdiction where token sales, crypto-asset activities, or participation in similar projects are restricted or prohibited, or where such participation would require registration or licensing not obtained; (c) you are not a person or entity on, or controlled by persons on, any sanctions or restricted-party lists, and you are not subject to export-control restrictions; and (d) you are of legal age and have the full capacity to participate.

Deemed acknowledgments, representations, and warranties

By accessing or relying on any part of the Materials, you are deemed to acknowledge, represent, and warrant to the Organization, any Distributor, the Team, and their affiliates that:

1. You will not rely on the Materials in making any purchase, sale, investment, or other decision; you will rely only on your own advisors and, where applicable, the Definitive Terms.

2. URI may have no monetary value; there is no guarantee or representation of liquidity, transferability, price, demand, or market for URI.

3. URI is intended solely to enable usage of, and interaction within, UriNet as described (and as may be changed); URI is not designed for speculative investment, profit expectations, or passive income.

4. None of the Organization, the Team, any Distributor, nor their affiliates are responsible for the value, transferability, or availability of any market for URI, whether on third-party exchanges or otherwise.

5. You understand and accept the risks described in the Risk Factors section below and any additional risks that may be set out in other project documentation.

Token characterization; no rights

URI is intended to be a non-refundable, functional, utility token used within UriNet. URI does not represent or confer: (i) any equity, ownership, shareholding, security, debt, or equivalent rights; (ii) any rights to dividends, distributions, profits, interest, or other returns; (iii) any governance, voting, or management rights in the Organization or its affiliates (except, if implemented, on-protocol voting mechanisms that may affect certain technical parameters of UriNet and which may be modified, curtailed, or discontinued at any time); (iv) any claim, proprietary interest, or entitlement to intellectual property; or (v) any right to receive financial statements or other corporate information.

URI is not money, legal tender, electronic money, negotiable instrument, deposit, investment contract, commodity, derivative, or any other regulated financial product. URI is not a loan to the Organization or any other person.

Token functionality (illustrative only; subject to change)

Subject to implementation and change, URI may be used to (a) pay for network usage (e.g., transaction execution, bandwidth, storage, or other metered resources), (b) participate in protocol-level mechanisms (e.g., staking or delegation to support network security, if implemented), and (c) access or interact with applications, services, or modules deployed on UriNet. Any rewards or incentives (if any) are not guaranteed, may be modified or terminated, and may be subject to additional eligibility, lockups, slashing, or clawback conditions.

Secondary markets; independent exchanges

To the extent a secondary market for URI develops, it is operated by independent third parties. The Organization and the Team do not operate, control, or endorse any exchange or marketplace, provide any listing, market-making, or liquidity support, or make any representations about availability or legality of trading URI in any jurisdiction.

Informational purposes only; no reliance; no duty to update

The Materials describe potential features and a high-level technical and product direction. They are conceptual and may change substantially. Do not rely on the Materials for purchasing,

investment, accounting, or tax decisions. The Organization has no obligation to update, correct, or provide continued access to the Materials.

Regulatory status; approvals

No regulator has examined, approved, or endorsed the Materials or URI. Publication, distribution, or dissemination of the Materials does not imply compliance with applicable laws, rules, or regulations.

Forward-looking statements

The Materials may include forward-looking statements, including statements about plans, objectives, projections, future events, or performance. These statements are subject to numerous known and unknown risks and uncertainties that could cause actual results to differ materially. You should not place undue reliance on forward-looking statements. The Organization undertakes no obligation to update such statements.

Risk factors (summary, non-exhaustive)

Participating in UriNet or acquiring/holding/using URI involves significant risks, including but not limited to: (a) protocol, software, or smart-contract defects; (b) cryptographic advances or attacks; (c) validator or network failures, congestion, forks, governance disputes, or chain halts; (d) cybersecurity incidents, key loss, phishing, and operational errors; (e) volatility, illiquidity, or loss of value; (f) regulatory, tax, and compliance changes or enforcement; (g) market, economic, or business risks; (h) reliance on third-party infrastructure (wallets, exchanges, oracles); (i) adverse outcomes from upgrades, parameter changes, or governance actions; and (j) force majeure events. This summary is not exhaustive; additional risks may be described in other documentation, including the Definitive Terms.

Taxes

You are solely responsible for determining, reporting, and paying any taxes arising from your activities related to UriNet or URI, and for maintaining all required records. The Organization does not provide tax advice.

No fiduciary duties; no advice

The Organization and the Team owe you no fiduciary duties, and no communication or information constitutes investment, legal, accounting, or tax advice. You should obtain independent professional advice as needed.

Intellectual property; third-party references

All trademarks, service marks, trade names, logos, and content are the property of their respective owners. References to third-party projects, companies, or platforms are for illustrative purposes only and do not imply affiliation, endorsement, or sponsorship.

Language; translations

This Whitepaper may be translated. The English version controls in the event of any conflict or ambiguity. By accessing the Materials, you acknowledge that you have read and understood the English version.

Privacy; KYC/AML

Any collection or processing of personal data (including for KYC/AML/sanctions screening, if applicable) will be conducted pursuant to the project's privacy policy and applicable law. Participation may require you to provide information and documentation for compliance purposes; failure to do so may result in denial or termination of access.

No distribution; restrictions on use

No part of the Materials may be copied, reproduced, redistributed, or disseminated, in whole or in part, without the prior written consent of the Organization. Access to the Materials may be restricted in certain jurisdictions. You are responsible for ensuring lawful access.

Limitation of liability; disclaimer of warranties

TO THE MAXIMUM EXTENT PERMITTED BY LAW, THE MATERIALS AND ANY SOFTWARE OR DOCUMENTATION ARE PROVIDED "AS IS" AND "AS AVAILABLE," WITHOUT WARRANTIES OF ANY KIND, WHETHER EXPRESS, IMPLIED, OR STATUTORY. TO THE

MAXIMUM EXTENT PERMITTED BY LAW, IN NO EVENT SHALL THE ORGANIZATION, THE TEAM, ANY DISTRIBUTOR, OR THEIR AFFILIATES BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, CONSEQUENTIAL, EXEMPLARY, OR PUNITIVE DAMAGES, OR FOR ANY LOSS OF PROFITS, REVENUE, DATA, GOODWILL, OR USE, ARISING OUT OF OR RELATED TO THE MATERIALS, URINET, URI, OR ANY ACTIVITIES CONTEMPLATED HEREIN, WHETHER IN CONTRACT, TORT (INCLUDING NEGLIGENCE), STRICT LIABILITY, OR OTHERWISE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.